

CPSC 375 High-Performance Computing

Lecture 23

The Sieve of Eratosthenes

Algorithm

The Sieve of Eratosthenes

An ancient algorithm that identifies all prime numbers up to a specified limit by iteratively marking the multiples of each discovered prime, starting from 2, as composite.

1. Create a list of unmarked natural numbers $2, 3, \dots, n$
2. $k \leftarrow 2$
3. Repeat
 - (a) Mark all multiples of k between k^2 and n
 - (b) $k \leftarrow$ smallest unmarked number $> k$until $k^2 > n$
4. The unmarked numbers are primes

Sequential Algorithm

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39	40	41	42	43	44	45	46
47	48	49	50	51	52	53	54	55	56	57	58	59	60	61

Complexity: $\Theta(n \ln (\ln n))$
Impractical for large n

Sources of Parallelism

- Domain decomposition
 - Divide data into pieces
 - Associate computational steps with data
- One primitive task per array element

Parallel Algorithm

Step 3(a)

Mark all multiples of k between k^2 and n

```
for all  $j$  where  $k^2 \leq j \leq n$  do
  if  $j \bmod k = 0$  then
    mark  $j$  (it is not a prime)
  endif
endfor
```

Parallel Algorithm

Step 3(b)

Find smallest unmarked number $> k$

Min-reduction (to find smallest unmarked number $> k$)

Broadcast (to get result to all tasks)

Agglomeration Goals

- Consolidate tasks
- Reduce communication cost
- Balance computations among processes

Data Decomposition Options

- Interleaved (cyclic)

e.g.

process 0: 2, 2+p, 2+2p, ...

process 1: 3, 3+p, 3+2p, ...

- Easy to determine “owner” of each index
- Leads to load imbalance (why?)

Block Decomposition Options

- Balances loads – divide the array into p continuous blocks of roughly equal size.
- More complicated to determine the owner if n is not a multiple of p . e.g.,
 - e.g., Suppose $n = 1024$ and $p = 10$, which means $1024/10 = 102.4$.
 - If we give every process 102 elements, there will be four left over.
 - On the other hand, we cannot give every process 103 elements, because the array is not that large.
 - We cannot simply give the first $p - 1$ processes $\lceil n/p \rceil$ combinations and give the last process whatever is left over, because there may not be any elements left (see Homework).
 - Allocating no elements to a process is undesirable (why?).

Block Decomposition Options

- Want to balance workload when n not a multiple of p
- Each process gets either $\lceil n/p \rceil$ or $\lfloor n/p \rfloor$ elements
- Seek simple expressions
 - Find low, high indices given an owner
 - Find owner given an index

Method #1

- Let $r = n \bmod p$
- If $r = 0$, all blocks have same size
- Else
 - First r blocks have size $\lceil n/p \rceil$
 - Remaining $p-r$ blocks have size $\lfloor n/p \rfloor$

Examples

17 elements divided among 7 processes



17 elements divided among 5 processes



17 elements divided among 3 processes



Method #1 Calculations

- First element controlled by process i

$$i \lfloor n / p \rfloor + \min(i, r)$$

- Last element controlled by process i

$$(i + 1) \lfloor n / p \rfloor + \min(i + 1, r) - 1$$

- Process controlling element j

$$\min(\lfloor j / (\lfloor n / p \rfloor + 1) \rfloor, \lfloor (j - r) / \lfloor n / p \rfloor \rfloor)$$

Method #2

- Scatters larger blocks among processes
- First element controlled by process i :

$$\lfloor in / p \rfloor$$

- Last element controlled by process i :

$$\lfloor (i+1)n / p \rfloor - 1$$

- Process controlling element j :

$$\lfloor p(j+1) - 1 / n \rfloor$$

Examples

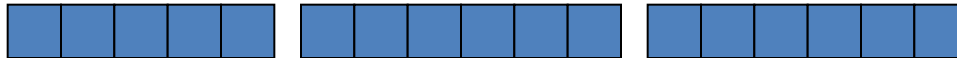
17 elements divided among 7 processes



17 elements divided among 5 processes



17 elements divided among 3 processes



Comparing Methods

Operations	Method 1	Method 2
Low index	4	2
High index	6	4
Owner	7	4

Assuming no operations for “floor” function

Pop Quiz

- Illustrate how block decomposition method #2 would divide 13 elements among 5 processes.

- First element controlled by process i :

$$\lfloor in / p \rfloor$$

- Last element controlled by process i :

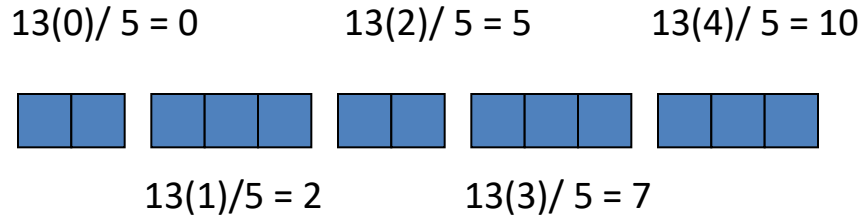
$$\lfloor (i+1)n / p \rfloor - 1$$

- Process controlling element j :

$$\lfloor p(j+1) - 1 / n \rfloor$$

Pop Quiz

- Illustrate how block decomposition method #2 would divide 13 elements among 5 processes.



Block Decomposition Macros

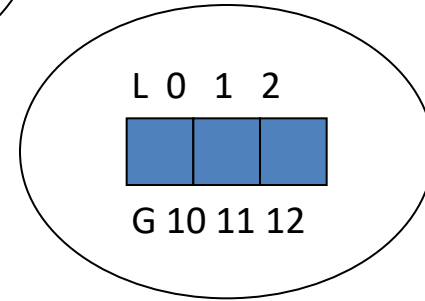
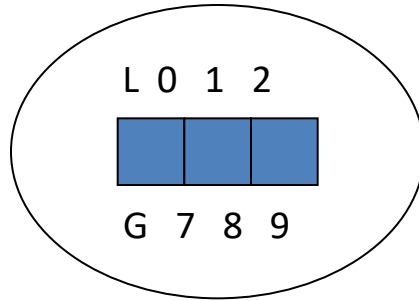
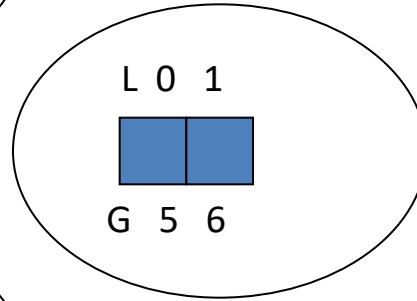
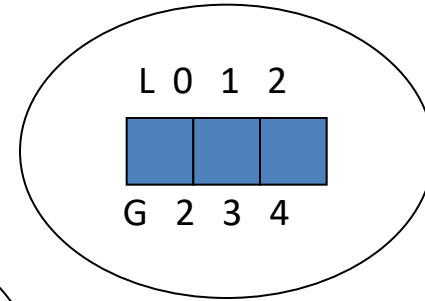
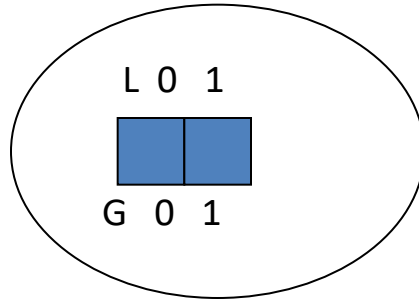
```
#define BLOCK_LOW(id,p,n) ((i)*(n)/(p))
```

```
#define BLOCK_HIGH(id,p,n) \
    BLOCK_LOW((id)+1,p,n)-1)
```

```
#define BLOCK_OWNER(index,p,n) \
    (((p)*(index)+1)-1)/(n))
```

```
#define BLOCK_SIZE(id,p,n) \
    (BLOCK_LOW((id)+1)- BLOCK_LOW(id))
```

Local vs. Global Indices



Looping over Elements

- Sequential program

```
for (i = 0; i < n; i++) {  
    ...  
}
```

- Parallel program

```
size = BLOCK_SIZE (id,p,n);  
for (i = 0; i < size; i++) {  
    gi = i + BLOCK_LOW(id,p,n);  
}
```

Decomposition Affects Implementation

- Largest prime used to sieve is $\sqrt{n}$
- First process has $\lfloor n/p \rfloor$ elements
- It has all sieving primes if $p < \sqrt{n}$ (reasonable assumption?)
- First process always broadcasts the next sieving prime
- No reduction step needed

Fast Marking

- Block decomposition allows the same marking as the sequential algorithm:

$$j, j + k, j + 2k, j + 3k, \dots$$

instead of

for all j in block

if $j \bmod k = 0$ then mark j (it is not a prime)

Parallel Algorithm Development

1. Create list of unmarked natural numbers 2, 3, ..., n

2. $k \leftarrow 2$

Each process does this

Each process creates its share of list

3. Repeat

Each process marks its share of list

(a) Mark all multiples of k between k^2 and n

(b) $k \leftarrow$ smallest unmarked number $> k$

Process 0 only

(c) Process 0 broadcasts k to rest of processes

until $k^2 > m$

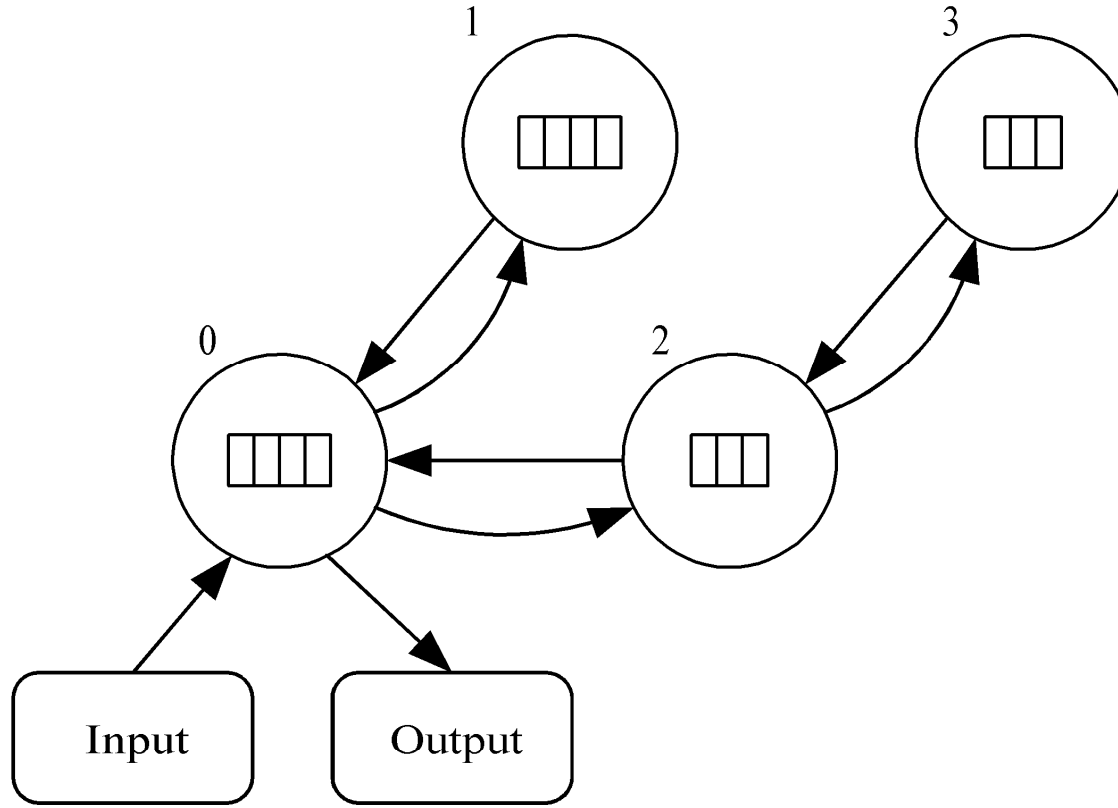
4. The unmarked numbers are primes

5. Reduction to determine the number of primes

Function MPI_Bcast

```
int MPI_Bcast (  
    void *buffer, /* Addr of 1st element */  
    int count,    /* # elements to broadcast */  
    MPI_Datatype datatype, /* Type of elements */  
    int root,     /* ID of root process */  
    MPI_Comm comm) /* Communicator */  
  
MPI_Bcast (&k, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

Task/Channel Graph



Analysis

- χ is the time needed to mark a cell
- Sequential execution time: $\chi n \ln (\ln n)$
- Number primes between 2 and n (Prime Number Theorem): $n / \ln n$
- Iterate up to $\sqrt{n}$. So, # loop iterations is approx.:
 $\sqrt{n} / \ln \sqrt{n}$
- Broadcast time (tree-like structure): $\lambda \lceil \log p \rceil$
- Expected execution time:

$$\chi n \ln \ln n / p + (\sqrt{n} / \ln \sqrt{n}) \lambda \lceil \log p \rceil$$

